

Groundwork

Turning scattered client input — recorded calls, chat exports, contracts, screenshots — into a plan you can defend, where every line is traceable back to the sentence somebody actually said.

Ashika · August 2026 · AI Business Discovery to POC

- | | |
|-------------------------------|-----------------------------|
| 1. Resources | 5. Assumptions |
| 2. What it does | 6. Limitations |
| 3. How the AI works | 7. The business perspective |
| 4. Running it, and testing it | |

This is a working tool, and deliberately not a finished product. The brief said the goal was to show how an open business problem becomes a working solution in limited time, so I built the parts that carry the idea and left the parts that only carry weight in production. Where I stopped, I have said so — sections six and seven are as honest as the rest.

ONE

Resources

REPOSITORY

<https://github.com/malicious28/groundwork>

Public. The history is worth a look: every feature was built on its own branch and merged separately, so `git log --graph --first-parent` reads as a sequence of decisions rather than one drop of code.

DOWNLOAD THE ZIP

<https://github.com/malicious28/groundwork/releases/download/v1.0/groundwork.zip>

A direct download — no account needed. Supplied as a link rather than an attachment because Gmail blocks any archive containing a JavaScript file, and one build config file is enough to trigger it.

VIDEO, PART 1

<https://www.loom.com/share/1c8385c26338433e8ff7b273c45a99c5>

VIDEO, PART 2

<https://www.loom.com/share/afb2c7b02aae4825ad0c332652abe097>

EXPLAINER

[docs/groundwork-explained.pdf](#)

19 pages in plain language, for a non-technical reader.

BUILD PLAN

[docs/groundwork-plan.html](#)

The plan and market survey written before any code.

TWO

What it does

A client never explains what they need in one clean document. It arrives as a recorded call, a WhatsApp thread, a PDF of their vendor terms, a screenshot of the spreadsheet they actually run on, and a paragraph in an email. Somebody has to read all of it and turn it into a plan — and that job fails in a specific way: details get lost, gaps get filled with guesses, and months later the client says *we never asked for that* with nobody able to prove otherwise.

Taking anything the client sends

Seven input types. The format is detected from the file's contents rather than its name, because clients rename things.

INPUT	HANDLING
Call transcripts	Speaker and timestamp preserved, so a quote can be attributed to a person
WhatsApp exports	Message-level, with sender and date
PDFs	Text extracted and split into citable sections
Word documents	Same, with headings kept as context
Screenshots	Read by a vision model at upload, not at analysis time
Web pages	Fetches by URL, with protection against internal addresses
Typed or pasted text	The most common thing a client actually sends

Screenshots deserve a note. A screenshot is not a document you can search, so it is transcribed the moment it is uploaded. That single choice means every later stage treats it exactly like a transcript, and a claim drawn from a spreadsheet screenshot can be quoted and verified like any other. Passing raw images into each analysis call would have put that content outside the checking path entirely.

What comes out

- 1 · Brief what they need, every line quoted and checked
- 2 · Contradictions where their own documents disagree with each other
- 3 · Questions what nobody has answered yet
- 4 · Process how it runs today, and what to change
- 5 · Outline roles, modules and features, prioritised
- 6 · Prototype a working demo of the tool being proposed

The rest of the product

FEATURE	WHAT IT IS FOR
Evidence ledger	Every source, split into addressable spans. The brief points back into this.
Grounding score	What proportion of the plan is backed by a real quote, shown rather than hidden.
Decisions	Settle a contradiction and it is fed into the next run, so it is not raised again.
Answers	Record what the client said and the next brief stops treating it as a gap.

Teams	Invite colleagues into a workspace; everyone adds to the same evidence.
Client link	A shareable link with no account, showing the plan and the demo — and none of the internal notes.
Proposal export	A document the client can keep, editable before it goes out.

THREE

How the AI works

Six calls, not one

Pressing *Run discovery* makes six separate requests. Each is a different job, and a single request asking for everything does every part of it worse. They also fail independently — a broken prototype does not cost you the brief — each is versioned separately so you can re-run one thing, and the reader watches the work arrive instead of staring at a spinner.

Every call is told the exact shape of answer it must return, so what comes back is structured data rather than prose to be parsed hopefully. The five calls after the brief do not depend on each other, so they run at the same time.

The idea the whole thing rests on

AI makes things up. That is the known problem, and every serious objection to using it in consulting work comes back to it. So here, the AI writes the plan but **it does not get to be believed**.

Every statement must carry a verbatim quote from a source document. After the model answers, the application goes back to the original text and checks the quote is really there: first an exact match, then a match that ignores whitespace and punctuation differences, then a fuzzy match that is recorded but deliberately *not* counted as verified. Only the first two count.

The result is a score, shown at the top of the brief. Clicking any citation opens the source with the sentence highlighted in place.

84%

seeded demo — 27 of 32

89%

a live run — 40 of 45

197

quotable spans, 6 documents

230

tests, plus 12 in a browser

It is deliberately not 100%

Four of the unverified statements are honest assumptions, marked as assumptions. The fifth is a claim planted on purpose: it cites a sentence that appears in no document anywhere, and it renders red every time.

That is the proof the check can fail. A verification layer that always says yes is decoration. Tests assert both ends — that the real citations pass, and that the planted one does not.

Two things it catches that people miss

Contradictions. The founder says the first-phase budget is two lakh; two weeks later the head of operations says five. Both are in the notes, nobody noticed, and a plan built on the wrong number fails. It shows both sides with a quote each and does not pick a winner, because that is the client's decision.

Unanswered questions. Budget, timeline, integrations, what happens to eight months of spreadsheet history on the day the new system goes live. Each phrased so it can be pasted straight into an email. An absent answer is a finding; most tools quietly guess.

The demo it builds

The sixth call does not describe what to build — it builds a working demo. A single self-contained page seeded with the client's real project names and people, where actions have consequences that cross screens: a supervisor posts an update and it appears on the client's view; post something mentioning a cost and it stops in the founder's approval queue instead. It runs inside a sandbox that can execute its own code but cannot read anything from the application around it.

The client does not just read the proposal. They can use it, and tell you it is wrong before anyone writes production code.

FOUR

Running it, and testing it

No Docker, no database server, no cloud account, and no API key needed to see everything working. The database is Postgres compiled to WebAssembly, running inside the application process. Node 20 or newer is the only requirement.

From the repository

```
git clone https://github.com/malicious28/groundwork.git
cd groundwork
```

From the zip

```
# download, then
unzip groundwork.zip
cd groundwork
```

Then, either way

```
npm install
cp .env.example .env

# AUTH_SECRET is required — it signs the session cookie. Generate one:
node -e "console.log(require('crypto').randomBytes(48).toString('base64url'))"
# paste the output after AUTH_SECRET= in .env
```

```
npm run setup      # creates the database, applies security policies, loads the demo
npm run dev        # http://localhost:3000
```

The sign-in screen carries the credentials, so there is nothing to look up. All three accounts use the password `demo1234`.

ACCOUNT	WHAT IT SHOWS
<code>ashika@meridian.example</code>	The consultant. The full workspace with the demo engagement.
<code>rohit@novainteriors.example</code>	Their client. Only what was shared, read-only.
<code>dev@northwind.example</code>	An unrelated firm. Proof that it sees none of the above.

A five-minute test

1. Sign in as the consultant. The dashboard shows six documents and 197 quotable spans.
2. Press **Run discovery** and watch the six stages land.
3. Open **Brief**. Note the score. Click any green citation — the source opens with the sentence highlighted.
4. Find the red citation on the claim about Tally. It cites a sentence in no document. This is the planted claim, and it is the point of the whole tool.
5. Open **Conflicts** and record a decision on the budget disagreement.
6. Open **Prototype**. Post an update from site, then switch to the client view and watch it appear. Post one mentioning a cost and watch it queue for approval instead.
7. Sign out, sign in as the client. You now see only what was shared — no contradictions, no internal notes.
8. Sign in as the third account. The demo project is not merely hidden; opening its URL directly returns a 404.

Testing it the other way

```
npm test          # 230 tests
npm run test:e2e  # 12 tests driving a real browser through the demo
npm run db:fresh  # an empty workspace, to walk the flow with your own documents
npm run db:reset  # back to the demo – the fastest fix for most problems
```

Using a real API key

Optional. Without one, the app replays a recorded analysis of the seeded project — and the verification layer still runs for real against the actual documents, which is why the demo can show a claim failing its check. With a key, the same code path calls the model. `npm run set-key` writes it into `.env` correctly, and the server must be restarted because `.env` is read once at startup. A full live run takes about eleven minutes.

FIVE

Assumptions

Stated plainly, because most of them could reasonably have gone the other way.

About the problem

- **"Scattered inputs" means formats, not volume.** A handful of documents in incompatible formats, not thousands of files. That ruled out a vector database: the whole corpus fits in one context window, and chunking it would make cross-document synthesis worse rather than better. At a few hundred documents this reverses.
- **The reviewer's time is the scarcest resource.** Hence a demo that runs with no API key, no Docker and no cloud account, and a seeded engagement that shows the whole flow without anyone uploading anything.
- **A prototype they can operate beats one they look at.** The brief says "basic POC". I read that as something the client can carry the proposed workflow through, which is why the generated demo has working state and consequences that cross screens.

About the product

- **A consultant will not trust output they cannot check.** Everything rests on this. It is why every claim carries a quote that is re-checked, and why the score is shown.
- **A wrong claim is worse than a missing one.** An unverifiable claim is shown as unverified rather than quietly dropped, and gaps are raised as questions rather than filled with plausible guesses.
- **Clients read what you send them and nothing else.** The client view and the exported proposal deliberately omit the contradictions — quoting their own staff disagreeing with each other is a working note for the consultant.

About the technical setting

- **Single database, multi-tenant, small scale.** Row-level security in one Postgres is the right shape for tens of workspaces. A database per tenant is right at a different scale and is not this.
- **No email service.** Invitations produce a link you send yourself. Adding a provider is an afternoon and would prove nothing about what is being assessed.
- **The model changes its mind between runs.** Hence versioned artifacts, nothing overwritten, and a comparison view — because two runs over the same corpus genuinely differ.

SIX

Limitations

Deliberately not built

NOT BUILT	WHY, AND WHAT IT WOULD TAKE
Deployment	Configured for Vercel; it has only ever run locally. The configuration is in the repository.
Background jobs	Analysis runs inside the request, so closing the laptop mid-run loses the stages still in flight. Finished stages are saved. A queue is the fix.
Email delivery	Invitations produce a link to send yourself.
File storage	Uploads are parsed to text and stored in the database; the original file is not kept, apart from screenshots.

Rate limiting, audit log	Both matter in production. Neither proves anything about what is being assessed.
Retrieval over large corpora	Not needed at this size, and it would make synthesis worse. Necessary past a few hundred documents.

Where the design could be wrong

- Verification is exact-match with a normalised and a fuzzy fallback. A model that paraphrases well but quotes loosely scores badly here even when it is right. I would rather that than the reverse.
- Prompt caching is set up and, measured, never pays off: the output schema sits ahead of the system prompt in the cached prefix, and the six stages each need their own schema. Making it work would mean one shared schema validated only after the fact. I kept the per-stage schemas, because a guaranteed-shape response is what makes the output checkable. Somebody optimising for cost would decide the other way.
- The recorded analysis only describes the seeded project and refuses to run against anyone else's documents. Deliberate, but it does mean the no-key demo is a demo rather than a trial.

What can go wrong when you run it, and why

All of these say the same thing in the application. None fails silently.

MESSAGE	CAUSE AND FIX
No API key is set	No key, or a key added while the server was running — <code>.env</code> is read once at startup. Run <code>npm run set-key</code> and restart, or ignore it and use the demo.
Account has no capacity left	The key has hit its usage limit. Nothing in the code can work around it; the demo still runs.
Answer was cut off after N tokens	A stage produced more than its budget allowed. Re-run — everything that finished is saved. Budgets were sized from measurement.
Database already open in process N	The embedded database allows one process at a time, usually a second <code>npm run dev</code> . Stop the other, or set <code>PGLITE_DATA_DIR</code> elsewhere. It refuses loudly because opening it twice corrupts the data silently.
Empty workspace, saving fails	A browser session from before <code>npm run db:reset</code> . Sign out and in.
Screenshot shows 0 spans	Reading an image needs a model, so with no key it is stored but not transcribed. Set a key and upload again.

SEVEN

The business perspective

What it is worth

Discovery is the part of a project hardest to charge for and most expensive to get wrong. The time goes on reading and reconciling; the risk sits in what gets missed. This does not replace the consultant — it removes the reading, and it produces evidence.

WHERE IT PAYS	HOW
Speed to proposal	Days of reading become minutes. A full analysis of six documents runs in about eleven minutes.
Fewer disputes	Every requirement is quoted back to its source. "We never asked for that" becomes a link rather than an argument.
Risk found early	Contradictions and gaps surface during discovery, when they are cheap.
Higher win rate	A prototype the client has used is a stronger sell than one they have watched.
Scope control	Assumptions are listed visibly, so the client corrects them at proposal stage rather than at delivery.

Who uses it

Three roles, enforced rather than cosmetic. The **consultant** owns the workspace and runs the analysis. Their **colleagues** are invited in and add to the same evidence. The **client** gets a link with no account, showing the brief, the proposed process and the working demo, with a proposal they can download.

How it scales

AXIS	WHAT HAPPENS
More customers	Multi-tenant from the first table. Separation is enforced twice — in the application, and again by policies inside the database under a role with no permission to bypass them. Adding a customer is a row, not an environment.
More load	The same schema runs on managed Postgres by setting one environment variable. The embedded database is a development convenience, not a different system.
More documents	The honest ceiling. Today the whole corpus goes in one context window, which is why cross-document synthesis works. Past a few hundred documents the answer is a retrieval layer feeding the same six calls — the verification layer would not change.
Cost	The model is configurable per deployment. The default is chosen because the work is careful reading rather than deep reasoning, and cost per run decides whether a consultant re-runs after every new document.

What I would build next

1. **Background jobs**, so a run survives a closed laptop and a single failed stage can be retried on its own.
2. **Client replies on the questions**, so answers arrive in the tool rather than in an inbox.
3. **Real invitation emails**.
4. **Retrieval**, once an engagement outgrows a single context window.

Happy to walk through any of this, or to go deeper on the verification layer and the tenant isolation — those are the two parts I would most want to be questioned on.